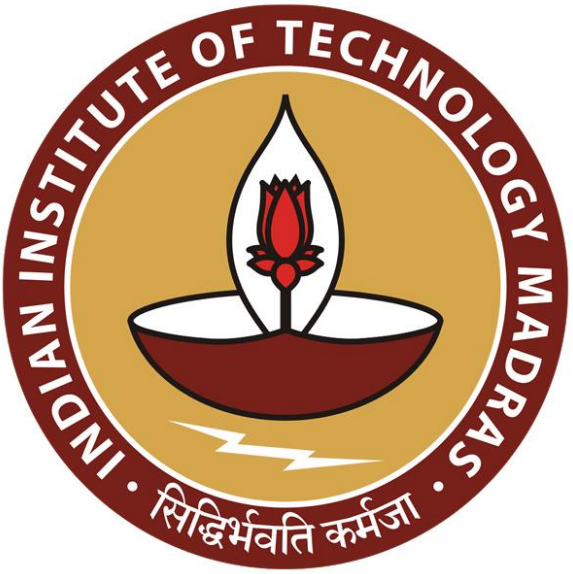
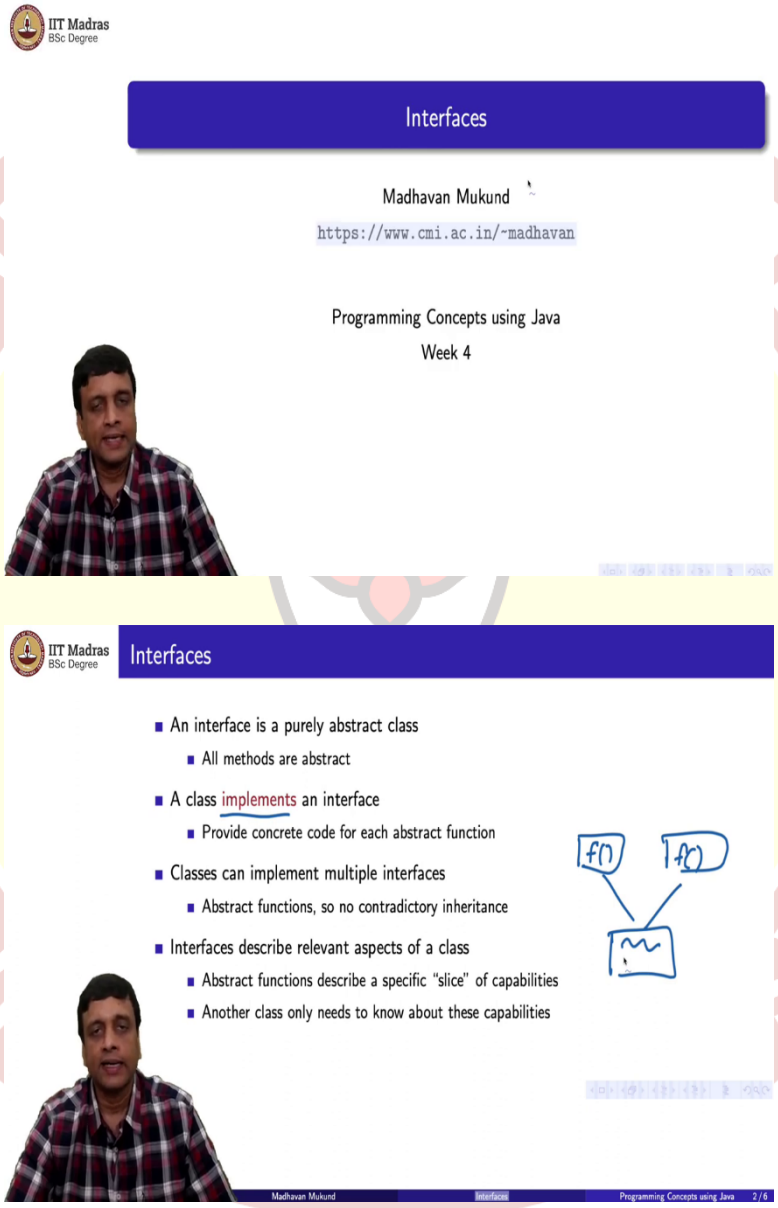


IIT Madras

ONLINE DEGREE

Programming Concepts Using Java
Professor Madhavan Mukund
Department of Computer Science
Chemical Mathematical Institute
Indian Institute of Technology, Madras
Interfaces

(Refer Slide Time: 00:14)



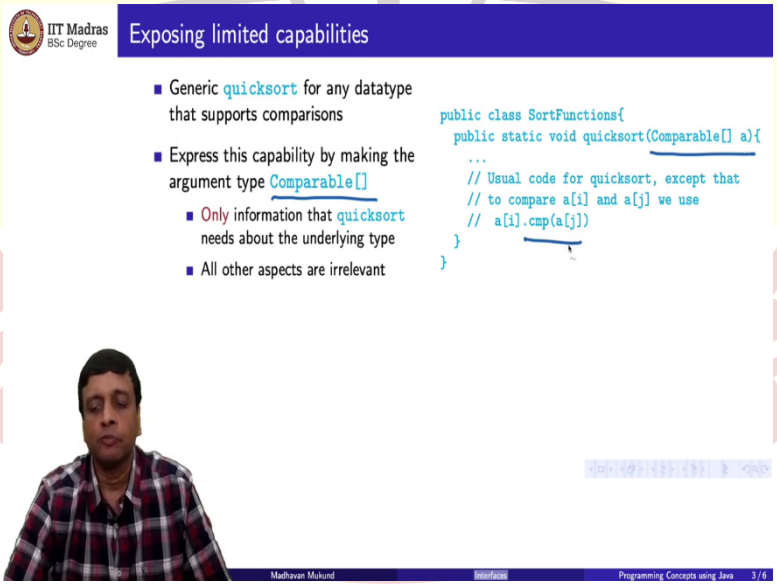
So, we saw last time that interfaces are a special type of abstract class. So, an interface is purely abstract. Inside an interface, everything must be abstract, in particular, all methods must be abstract. And since it is an abstract class, and we cannot create objects of it, there are no instance variables either.

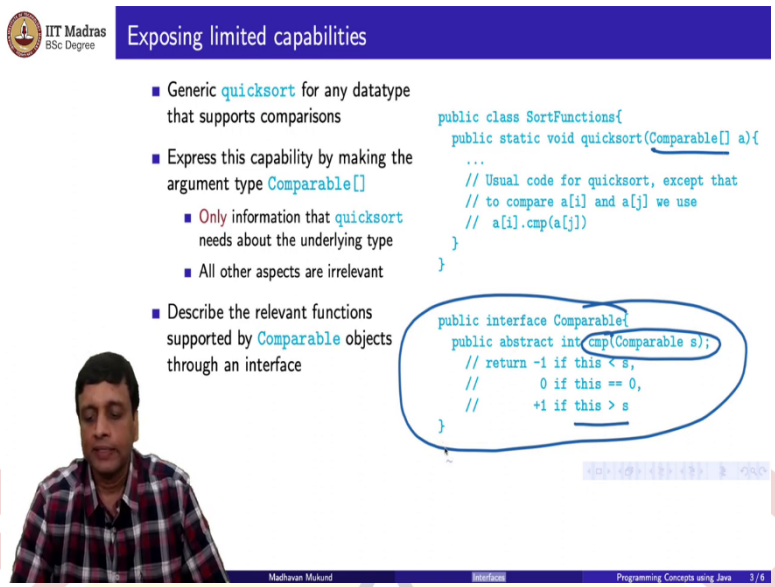
So, to distinguish between extending a class, and using an interface, we use this word implements. So, implements is the key word that Java uses. So, we are allowed to extend only one class, but we can implement multiple interfaces. So, one of the reasons for this for not allowing multiple inheritance that is to extend multiple classes is because we saw that you could have potentially a clash because the same function f() may be defined in two parent classes.

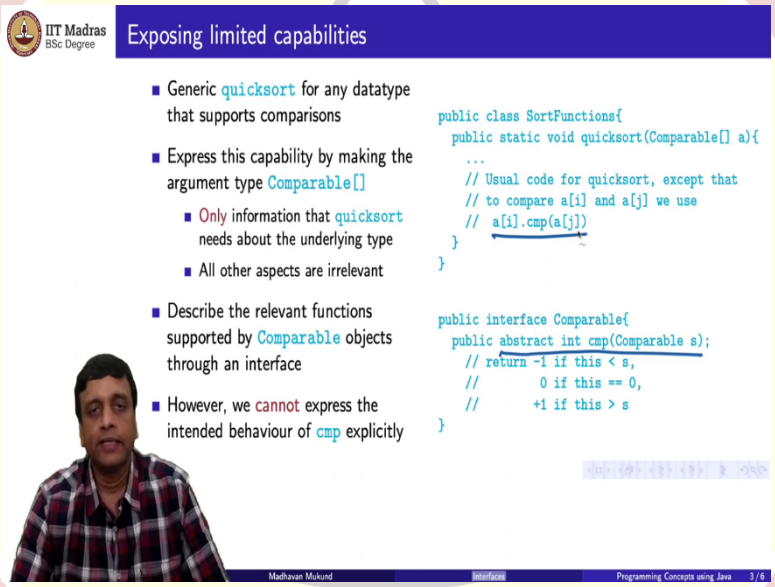
And then if you define a subclass which inherits from both, then there can be a contradiction. And then it is not clear how to resolve this contradiction, so you have to make some concessions for that. In an interface, this does not happen, because you do not have any concrete implementations, so presumably, you just have to implement it in the subclass.

So, the main use of interfaces is to describe a kind of capability. So, it describes what you might call a relevant aspect of a class. So, for instance, when we want to use some object and we need to know only a certain thing about it, then we can specify that certain capability that we need in terms of an interface.

(Refer Slide Time: 01:48)







So, the example that we saw last time is sorting. So, if we wanted to write a generic sort function, say, for example, quicksort, which will sort any data type, then what we really expect from quicksort or any sorting algorithm for that matter is that we need to be able to take a sequence a list or an array of items, and compare them so that we can arrange them in either ascending or descending order.

So, what is really key is that the data type that we are sorting supports comparisons, we do not really care. Other than that, whether they are doubles or floats, or integers, or complex objects, so long as we can compare them in some meaningful way. So, what we said was that we can abstract out this property of having a comparison function by specifying an interface. So, the interface we call comparable, and then, we can ask quicksort to take an array of any

object which is of type comparable, so that we can then compare these things using this cmp function, which is presumably defined at that interface comparable.

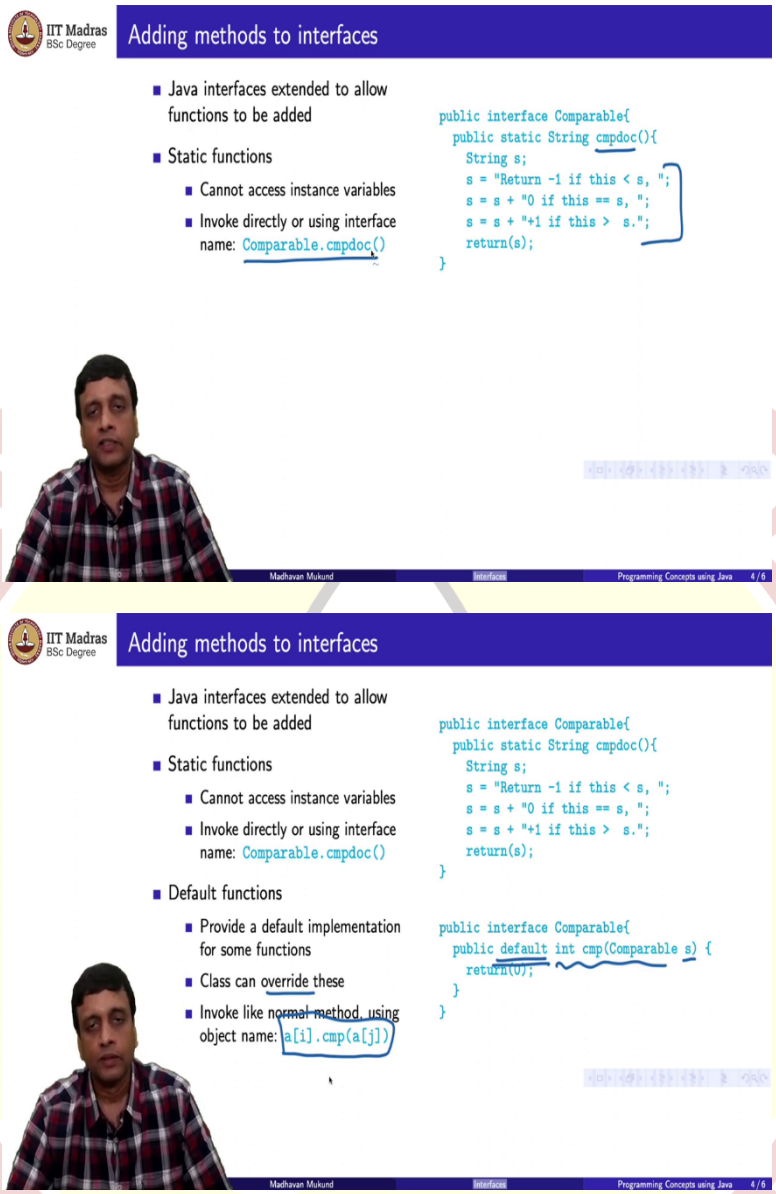
So, we have this interface comparable. And what it says is there should be an abstract function called cmp, which takes two objects, which implements interface and returns some convention. So, the convention is that you return a negative value typically -1, if the current object is smaller than the object that you are comparing with, it is 0 if the two objects are equal according to whatever size comparison you are doing. And it is 1 or a positive number if this object is bigger than that.

So, the idea is that you define the property that something can be compared through an interface called Comparable, and then you use that as the data type for the function that uses this property. So now, when I pass any array to quicksort, all I have to do is ensure that it maintains an interface. So, this is really the key capability, key property of an interface that it defines a capability.

Now, notice that it defines a capability in terms of the function that must be implemented. So, you basically get this signature here, you say, you need a function, which takes another object of the same type something else that implements Comparable, and returns an int that is all you know. So, the fact that, this specification is there, is not really provided by the interface, it is only something which must be documented externally. So, we have to say separately, that this interface is expected to return -1 if this is less than is equal to 0 and so on. But we cannot guarantee that an object that actually implements Comparable, faithfully follows this.

So, if that object does not implement cmp, in the way that we intend, then the outcome of the sort function will be unpredictable, and not according to our requirement. But that is something that we cannot really do. So, the interface can only tell you the signature, so that you can call that faithfully inside the function that uses that interface, but you cannot guarantee that the function that is being implemented actually does what you think it is doing. So, you have to take it on faith that the implementation obeys, whatever expectation is there of that function.

(Refer Slide Time: 04:59)



Now, as we said, interfaces were intended to be purely abstract objects. That is, you have some abstract function signatures, and no concrete instance variables. But over time, people found it convenient to be able to add some limited functions to an interface because otherwise everything that is in an interface has to be implemented by the calling class. So, there are some situations, maybe it is not really very crucial and you might want to allow some functions to be defined in the interface.

So, Java now allows a certain limited capability of defining functions in an interface. So, the first constraint is that the interface cannot be created as an object. So, we do not have instance variables. So, we can have without instance variables, static function. So, remember, a static

function is something that exists in the class without a variable or an object being created out of that class. So, we saw that for the basic function main that we need to start our Java. So just like that, in an interface, you can define a static function.

So here, for instance, supposing somebody is not sure what the cmp function should do, then you could have a static function inside this interface comparable, which is called cmpdoc(), which returns a string and the string basically tells you the documentation is return -1, if this is less than s and so on.

So, now this is a function, which is part of this interface, so it is like a static function. So, if it is in your path, if you are implementing an interface, you can just call it. And if it is, in some other interface, which you know about, then you can use the interface name and say it is like, if you remember, like System.out.println().

So, System is a class. So, we know that in System there is a stream called out and that out has a function called print ln, and so on. So, you can call it through the interface name, but it is a property of the interface because there are no objects. So, this is one thing that you can do now with Java interfaces that you can add static functions. And another thing you can do is to provide a default implementation of the abstract functions that you would like the interface to have.

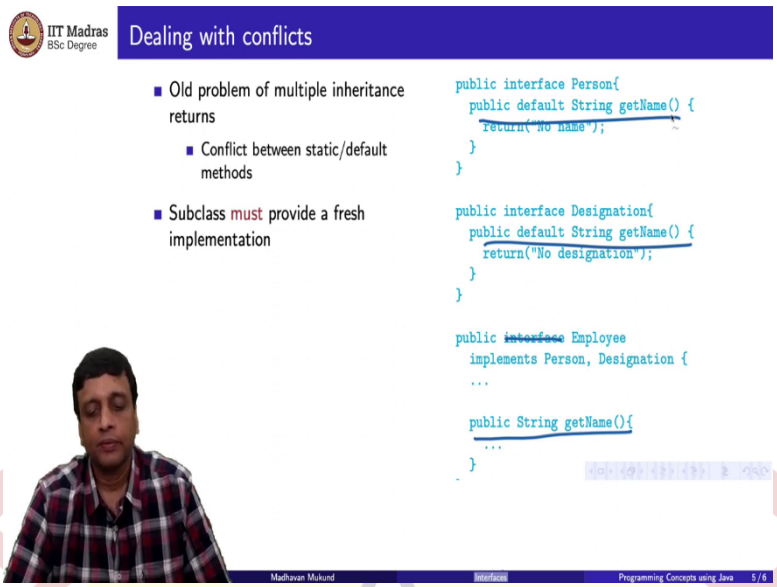
So, for example, suppose somebody forgets the implement comparable, but they do not have I mean, this is not a very good example, there may be other examples which are more meaningful, but supposing somebody does not implement cmp, then you can provide a default implementation of cmp. So, this one is one which always returns 0, for example. So, the word default here signifies that this is a default implementation, it has the same signature, and it is implicitly an expectation that this is an abstract function, which the deriving class, the class that implements this interface should reimplement, but should, if they do not reimplement it, then you get this function.

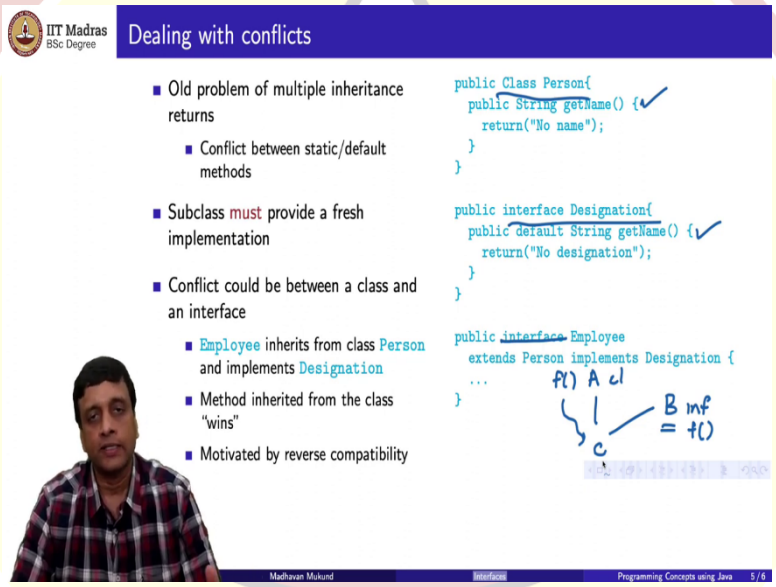
So, the class should typically override such function. So, their useful case is when there might be three or four functions in the interface. And maybe in some situations, some of those functions are not meaningful. So, then you could have classes which implement the interface but only implement a few other functions, and then the other ones will retain their default values, mainly, because you do not expect to call them in the context of such objects. So,

there are situations where these default values can be used. So, these are the two types of functions with Java allows you to associate now with an interface.

So, you have static functions and you have default functions. So, a default function basically is like the abstract function that the interface defines. So, for instance, this is a cmp. So, the way you would use it is like you would normally use cmp. So, you would use it in the context of an object. So, if you have a class which implements Comparable, and in that class, you use cmp, this is how we said you would use it in quicksort you use it in the same way, it is just that if you have not defined it, then it will pick up this default value. Now, you may have noticed that this has immediately reintroduced the same problem that we were trying to avoid with multiple inheritance of classes.

(Refer Slide Time: 08:50)





So, you could have now static or default definitions, which conflict across multiple interfaces that you are trying to implement. So, here is an example. So, you have an interface called Person and this has a default function called getName(), which returns a string “No name”. And you have another interface called Designation, which has another rival implementation as it were of getName() which says no designation.

And now you define a class, sorry, this should be a class. You define a class employee, which implements both these interfaces. So, this implements both these interfaces then which of these two definitions should it inherit? So, the rule that Java now follows is that it must actually give a fresh implementation.

So, this class must actually define getName itself because there are two conflicting getNames() that it is deriving from the two interfaces that it implements. So, this is the way the Java gets around this multiple inheritance problem in the context of these special functions that you can have with interfaces.

Now, you could have a slightly different type of conflict. So, this was a conflict between two interfaces, but of course, you could also extend a class and implement an interface. So, you extend a class. So, now person is a class and designation is an interface. So, now, if you extend a class and you implement an interface, and again you have these two functions, then again, implicitly, this public employee class inherits both.

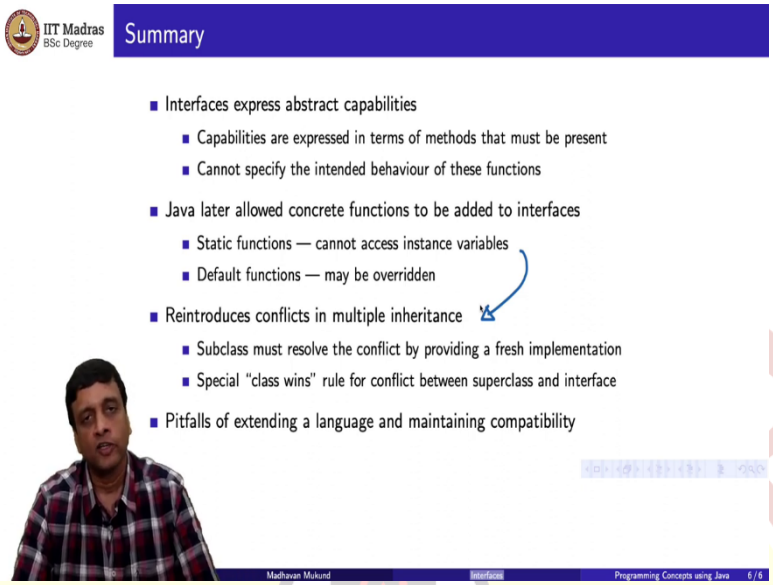
So here, if you have one coming from a class and one coming from an interface, Java's rule is the class will win. So, here you are not obliged to write it explicitly. And if you do not write, you do not kind of override this method inside employee, then what you will get is the one that you get from the superclass, not from the interface. In this case, the person function will get.

Now, the reason for this particular disambiguation is that before classes, before interfaces could have functions, people would have written something implementing an interface. Now, suppose somebody later on updates that interface to introduce a function, and this function happens to clash with a function that was already being derived. So, to avoid that code from crashing. So, you already have some, so you have a class C, which extends A and it, so this is an interface and this is a class.

So, you had written it at a time when B had no functions. And there was a function f() over here, which you were inheriting. And now somebody updates the interface B and adds a function f() here. So, the code for C is not expected to have known when it was written that B would eventually get a function called f() because at the time B, C was written, in fact, Java did not allow function.

So, to avoid this from creating errors in older code for a kind of legacy reverse compatibility, this disambiguation was done in the case where you inherit from one side from a class once a familiar interface. If you are inheriting from two interfaces, then presumably, you know what you are doing so you are forced to override.

(Refer Slide Time: 12:14)



So, to summarize, an interface expresses what you might call an abstract capability. But the only way that this abstract capability is actually described in terms of the signature of the methods that you need. So, you, for example, the comparable thing that we said, had a signature for a function called cmp.

Now, there is an intended behavior of cmp. How it tells you the comparison in terms of -1, and 0 and 1, but this cannot be explicitly defined. This is a convention and you can only hope that somebody who implements it knows the convention and writes their cmp function so that it respects what the interface is supposed to be. So, you can only specify the function should be there, and it must have this signature, but you cannot guarantee that the function actually behaves the way you want it to be.

So, Java has extended interfaces from being purely abstract to ones where you could have static functions without instance variables, and default functions, which could be overridden when you implement. And through this, Java has in some sense, reintroduced conflicts into the problem that it was trying to avoid by not allowing multiple inheritance of classes, because now you could have the same function name and the same signature coming from two sides, so there is a way of resolving this.

So, if your interface, if you are implementing two interfaces as a conflict you must override. If you are implementing an interface and extending a class then the class definition wins. But

this one could logically argue that you can go back now that you have done this, you maybe you can go back and for instance, allow multiple inheritance in classes also. So, once you go down the slippery slope, you might as well go all the way but then there are reasons to believe that multiple inheritance in general is not, I mean, very easy to program with.

So, Java has not gone that route. So, it has only allowed multiple inheritance of interfaces, but not of classes yet. But in general, you can see that Java is a language which is not static in its own definition. People have tried to extend it by adding new features, but then when you add a new feature, the problem is, you must maintain compatibility with the code that people have already written. So, this is perennially a problem with any system which grows over time. Whether it is a programming language or it is an operating system or it is any kind of piece of software that because of this requirement to have reverse compatibility with earlier behaviors.

Sometimes you cannot introduce the new features in the best possible way. You have to introduce it in a constrained way, so that it fits with what is what has been done before and does not break existing code. So, this is one of the pitfalls of kind of maintaining a language which is evolving, and at the same time keeping it compatible with earlier versions of that language.